

Image-based Approach to Reconstruct Curling in Continuum Structures

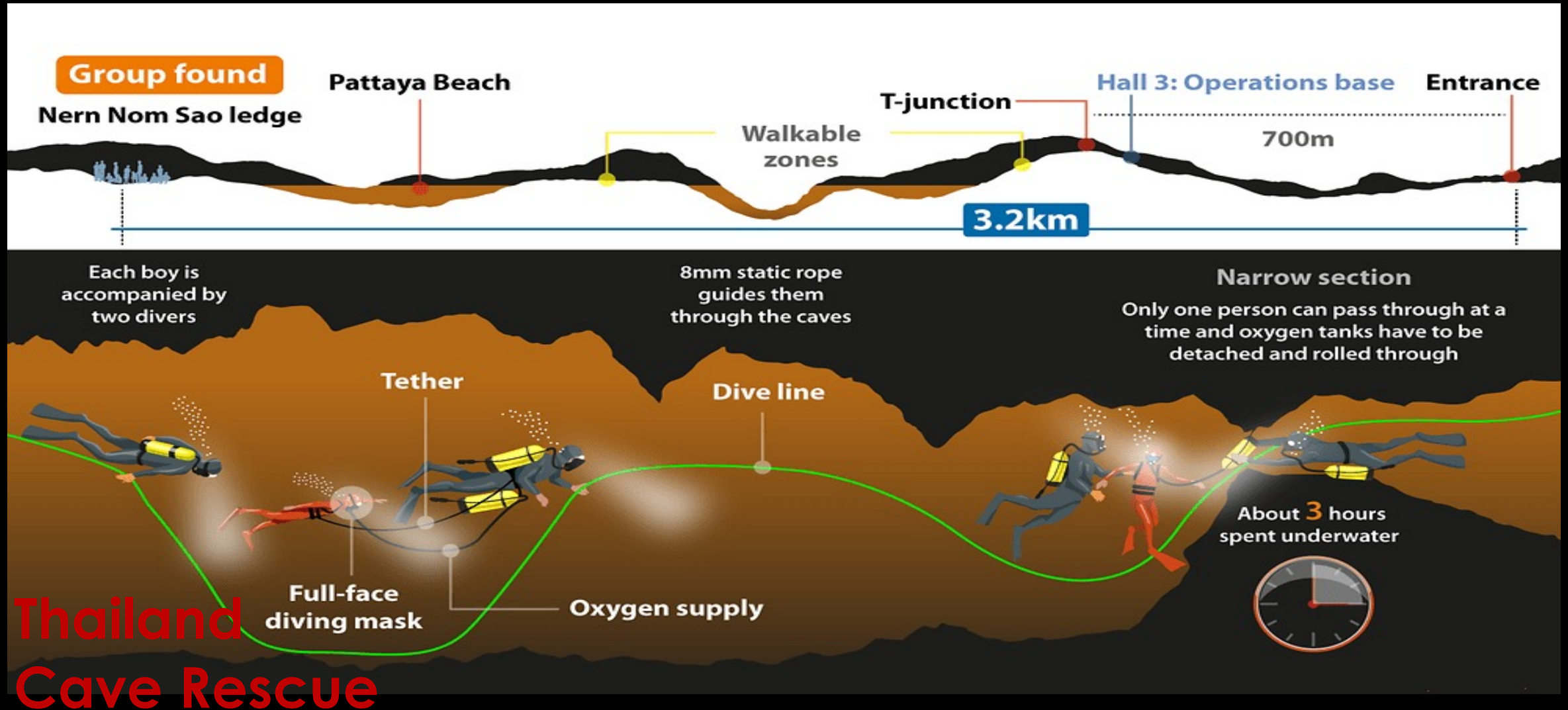
Jie Fan^{1,2}, Emanuela Del Dottore¹,
Francesco Visentin¹, Barbara Mazzolai¹

¹Center for Micro-BioRobotics, Istituto Italiano di Tecnologia, Italy

²The BioRobotics Institute, Scuola Superiore Sant'Anna, Italy

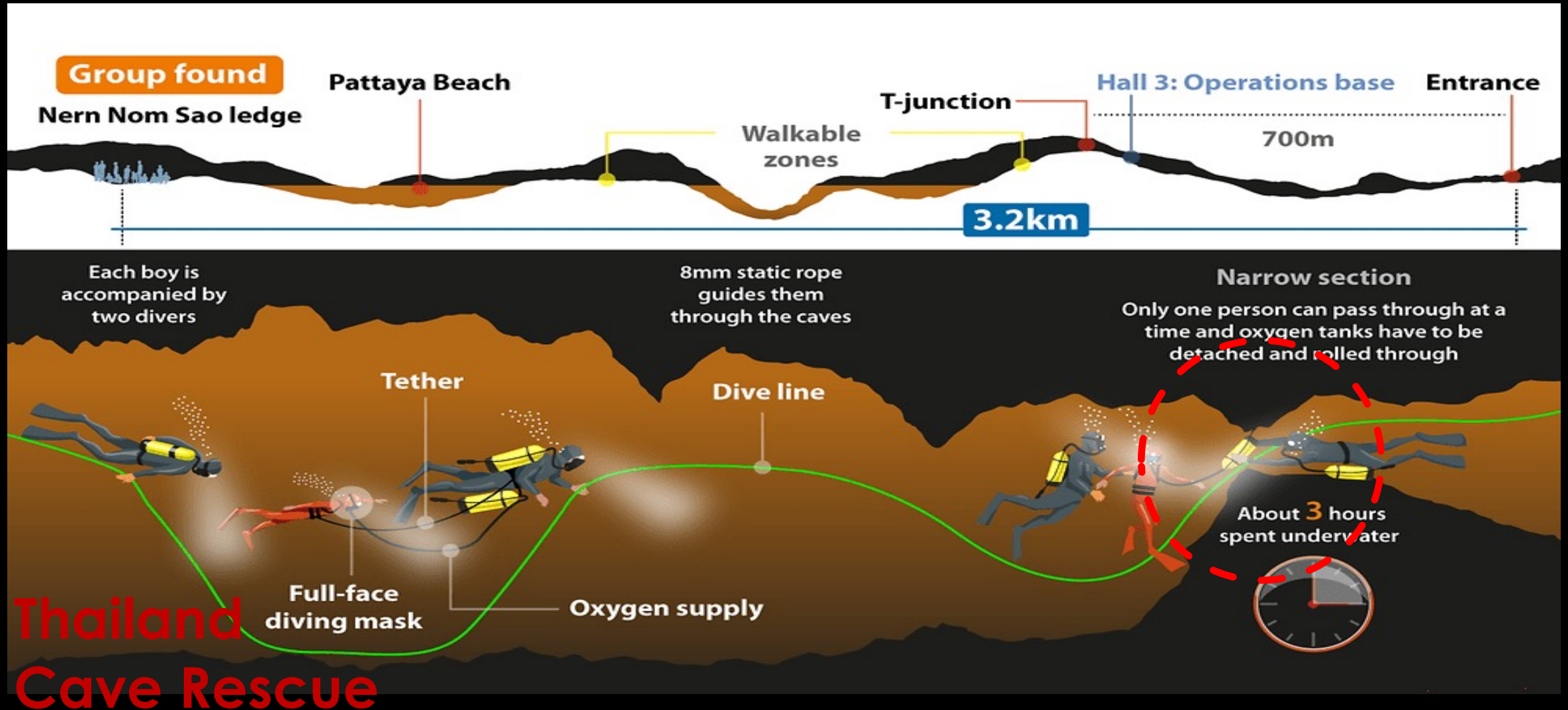
Motivation

Exploration of unstructured environment difficult to reach by humans



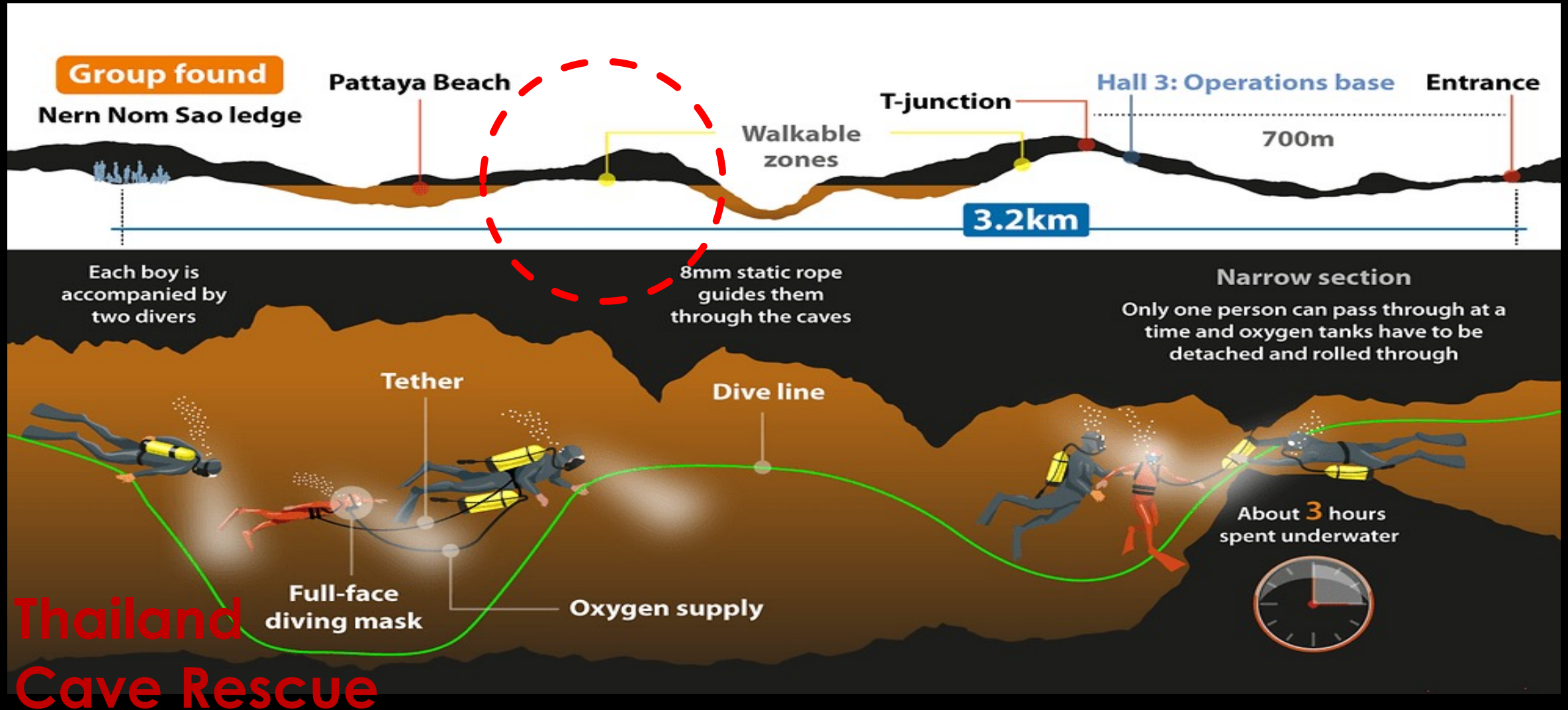
Motivation

Exploration of unstructured environment difficult to reach by humans



Motivation

Exploration of unstructured environment difficult to reach by humans



Lesson from Climbing Plants



The Private Life of Plants

<https://www.bbc.co.uk/programmes/p00lx6cl>

Lesson from Tendrils



Objective

Reconstruct and analyze filiform structures in nature (e.g. tendrils), to extract design rules for perception and motion in continuum plant-inspired robots



Tendril with support



Passionflower tendril

Proposed Approach

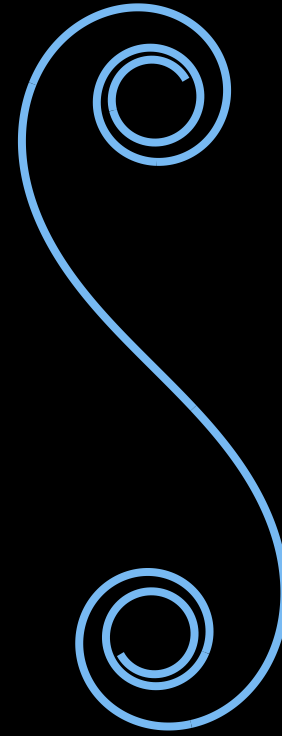
Image-based shape reconstruction with 2D clothoid model



Tendrill with support



Passionflower tendril



Clothoid curve

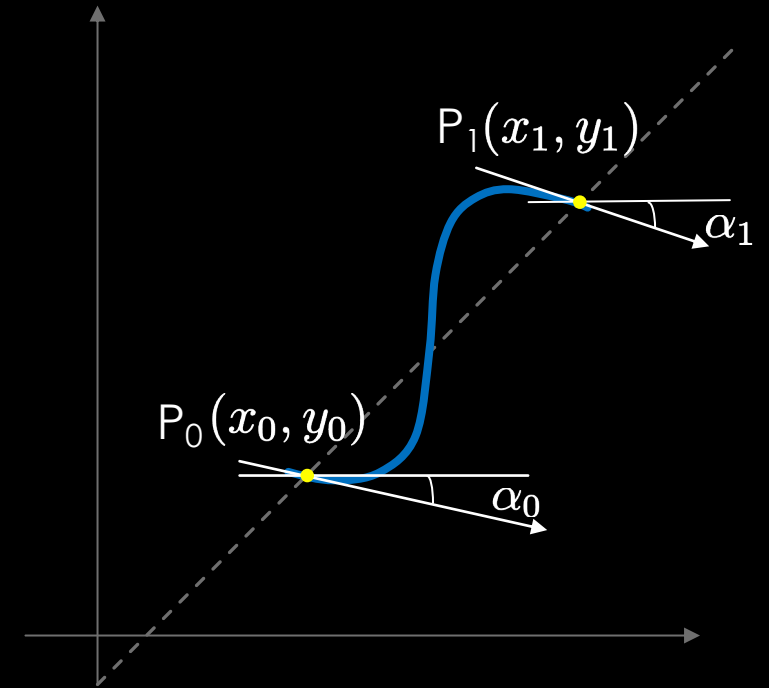
Why

Linear property allows a smooth transition of curvatures

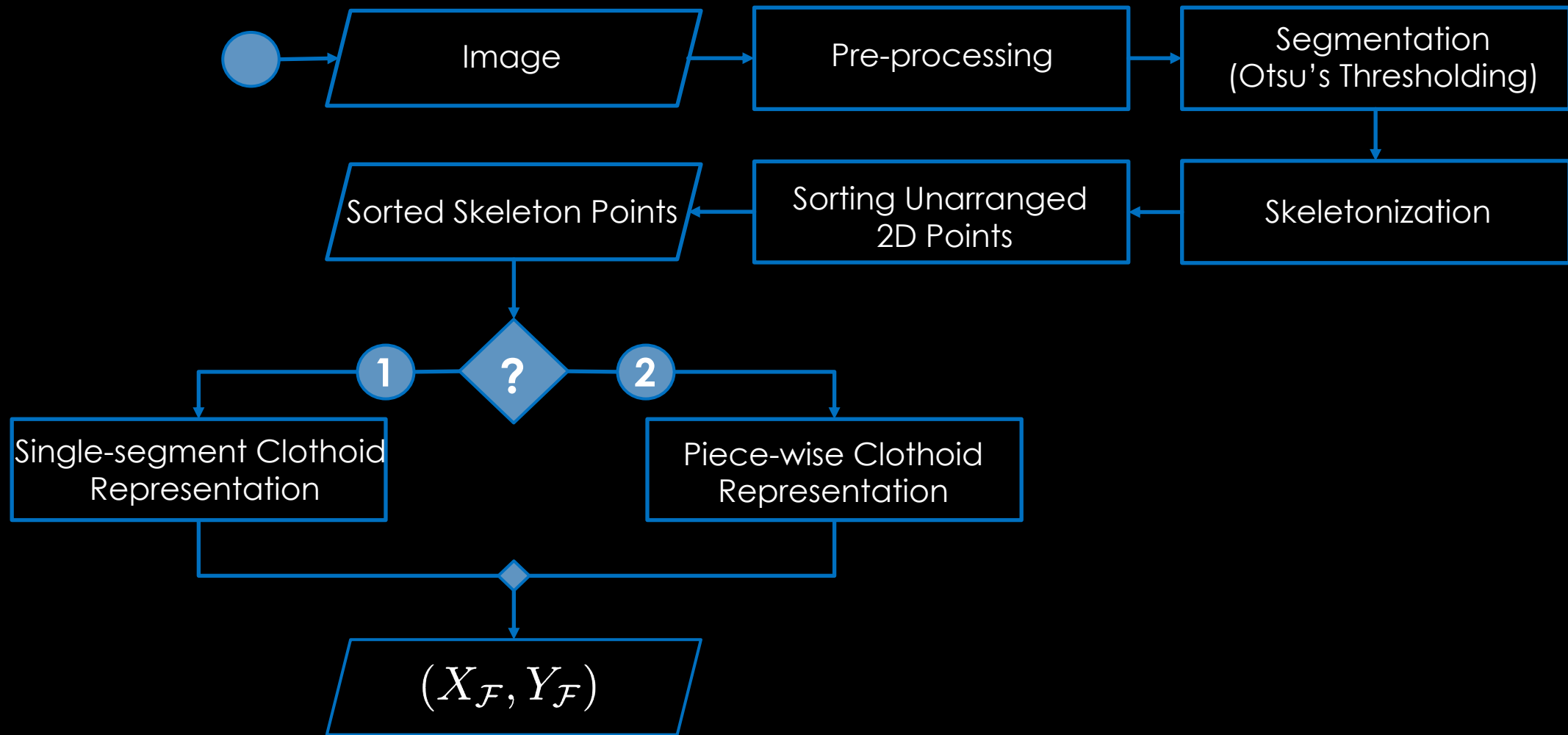
$$\begin{aligned}\mathcal{K}(s) &= k_1 s + k_2 \longrightarrow \text{clothoid} \\ k_1 &= 0 \longrightarrow \text{circle} \\ k_1 &= k_2 = 0 \longrightarrow \text{Straight line}\end{aligned}$$

Clothoid satisfies ODEs

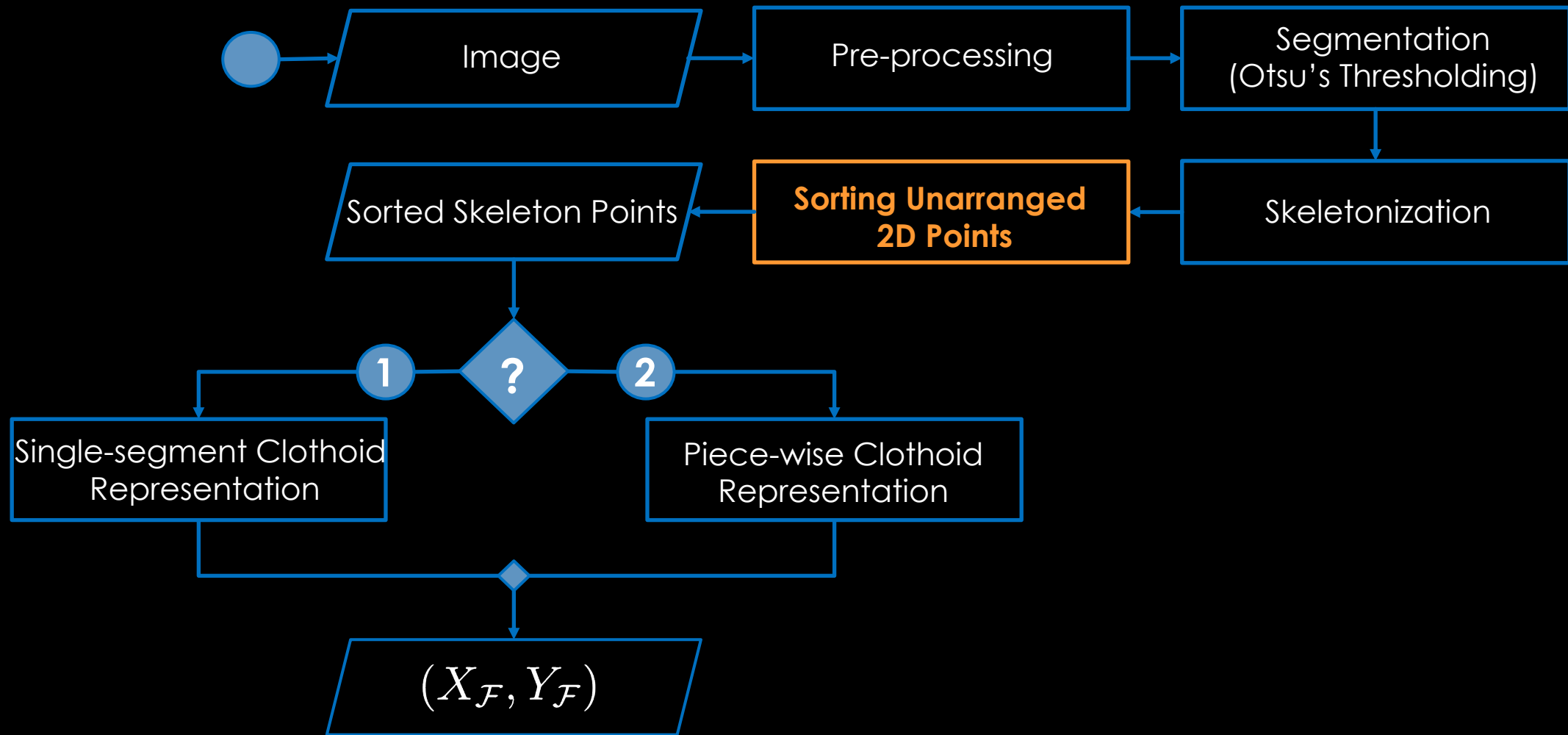
$$\begin{aligned}x'(s) &= \cos(\alpha(s)) & x(0) &= x_0 \\ y'(s) &= \sin(\alpha(s)) & y(0) &= y_0 \\ \alpha(s) &= \frac{k_1}{2}s^2 + k_2s + \alpha_0 & \alpha(0) &= \alpha_0 & \alpha'(s) &= \mathcal{K}(s)\end{aligned}\quad (\text{Eq.1})$$



Flowchart



Flowchart

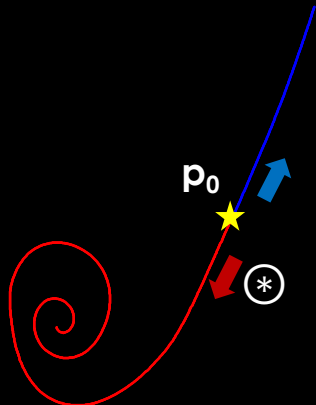


Sorting Unarranged 2D Points

Moving Least Square Method



Original



Start Point

Algorithm 3 Ordering points with a given direction $\otimes$ at point p_0 , given initial collecting distance d_0 , increment collecting distance d_ϵ , maximum break distance d_B .

Require: $\otimes, p_0, d_0, d_\epsilon, d_B, \mathcal{P}, d_M, \rho_0$

Ensure: $\mathcal{Q}_O$

```

1: function OrderingPoints( $\otimes, p_0, \mathcal{P}$ )
2:    $p_c = p_0, d_c = d_0, \mathcal{Q}_O = \emptyset, p_l = \emptyset$   $\triangleright$  Initialization.
3:   while  $d_c \leq d_B$  do
4:      $\mathcal{P}_+, \mathcal{P}_- = \emptyset, p_o = \emptyset$   $\triangleright$  Reset as empty.
5:      $\mathcal{P}_o, d_o, \overline{\mathcal{L}}_o \leftarrow \text{CollectNeighbouringPoints}(p_c, d_c, \mathcal{P},$ 
       $d_M, \rho_0)$ 
6:     for  $p_i$  in  $\mathcal{P}_o$  do
7:        $\overline{v}_c \leftarrow \overline{\mathcal{L}}_o$   $\triangleright$  Get direction vector of line  $\overline{\mathcal{L}}_o$ .
8:        $\overline{v}_i = p_c - p_i$   $\triangleright$  Get direction vector of line  $\overline{p_c p_i}$ .
9:       if  $\langle \overline{v}_i, \overline{v}_c \rangle \geq 0$  then  $\triangleright \langle \cdot, \cdot \rangle$  denotes scalar product.
10:         $\mathcal{P}_+ \leftarrow \mathcal{P}_+ \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_+$ .
11:      else
12:         $\mathcal{P}_- \leftarrow \mathcal{P}_- \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_-$ .
13:      end if
14:    end for
15:    if  $p_c == p_0$  &  $\otimes == \mathcal{A}$  &  $\mathcal{P}_- \neq \emptyset$  then
16:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_-} \|p_l, p_0\|$   $\triangleright \|\cdot, \cdot\|$  is Euclidean-distance.
17:    else if  $p_c == p_0$  &  $\otimes == \mathcal{B}$  &  $\mathcal{P}_+ \neq \emptyset$  then
18:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_+} \|p_l, p_0\|$ 
19:    end if  $\triangleright$  Ensure ordering direction at the first point.
20:    if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_+$  &  $\mathcal{P}_- \neq \emptyset$  then
21:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_-} \|p_o, p_0\|$ 
22:    else if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_-$  &  $\mathcal{P}_+ \neq \emptyset$  then
23:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_+} \|p_o, p_0\|$ 
24:    end if
25:     $d_c = d_o + d_\epsilon$   $\triangleright$  Increase collecting distance (no satisfied  $p_o$ ).
26:    if  $p_o \neq \emptyset$  &  $p_o \notin \mathcal{Q}_O$  then
27:       $\mathcal{Q}_O \leftarrow \mathcal{Q}_O \cup p_o$   $\triangleright$  Insert  $p_o$  to  $\mathcal{Q}_O$ .
28:       $p_l = p_c$   $\triangleright$  Update  $p_l$ .
29:       $p_c = p_o$   $\triangleright$  Update  $p_c$ .
30:       $d_c = d_0$   $\triangleright$  Reset collecting distance.
31:    end if
32:  end while
33:  return  $\mathcal{Q}_O$ 
34: end function

```

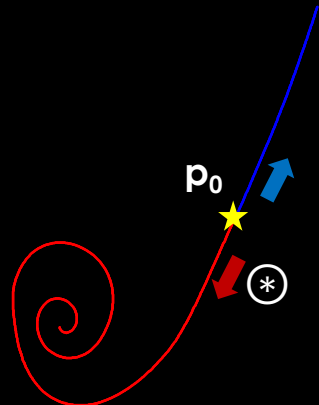
Sorting Unarranged 2D Points

Moving Least Square Method



Original

Collecting Neighboring Points



Start Point

Algorithm 3 Ordering points with a given direction $\otimes$ at point p_0 , given initial collecting distance d_0 , increment collecting distance d_ϵ , maximum break distance d_B .

Require: $\otimes, p_0, d_0, d_\epsilon, d_B, \mathcal{P}, d_M, \rho_0$

Ensure: $\mathcal{Q}_O$

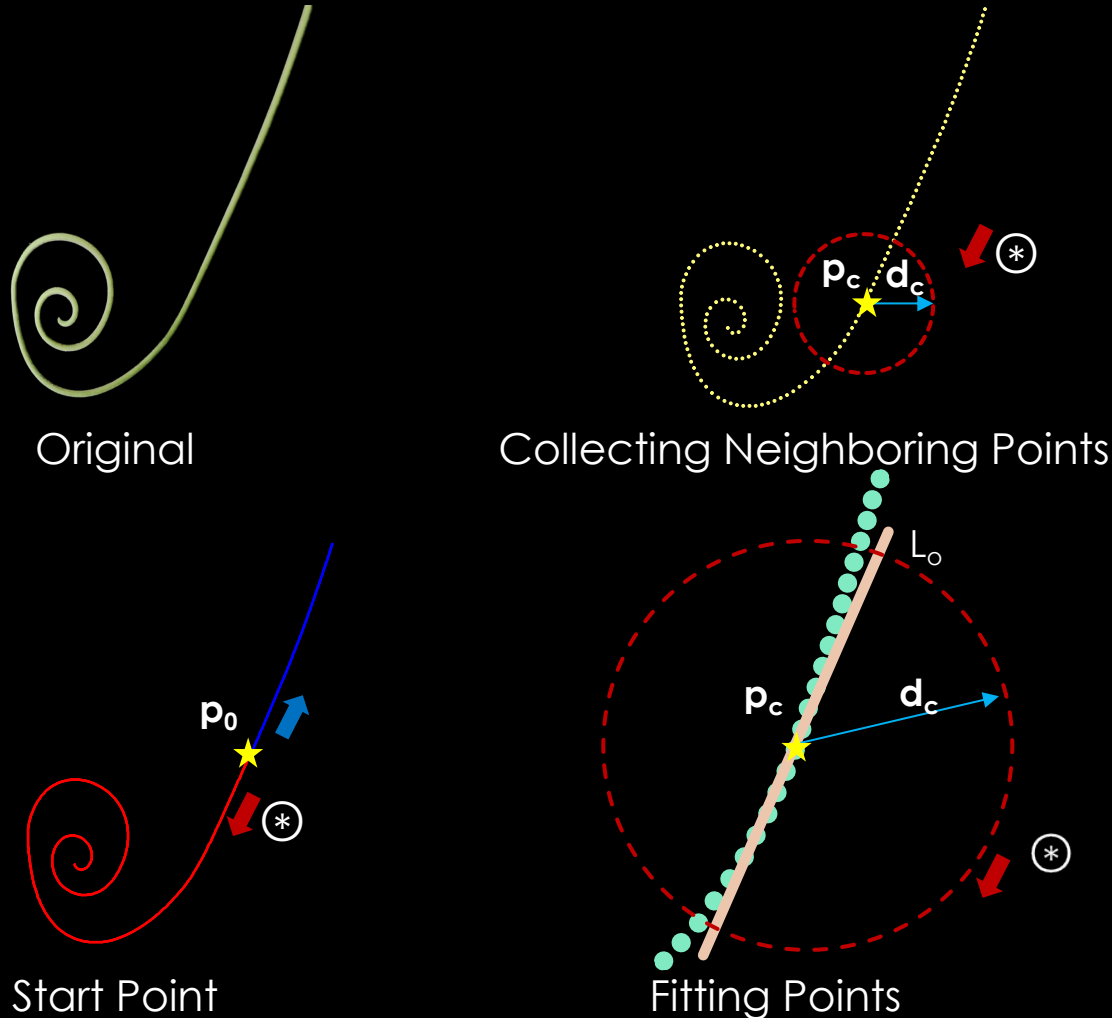
```

1: function OrderingPoints( $\otimes, p_0, \mathcal{P}$ )
2:    $p_c = p_0, d_c = d_0, \mathcal{Q}_O = \emptyset, p_l = \emptyset$   $\triangleright$  Initialization.
3:   while  $d_c \leq d_B$  do
4:      $\mathcal{P}_+, \mathcal{P}_- = \emptyset, p_o = \emptyset$   $\triangleright$  Reset as empty.
5:      $\mathcal{P}_o, d_o, \mathcal{L}_o \leftarrow \text{CollectNeighbouringPoints}(p_c, d_c, \mathcal{P},$ 
       $d_M, \rho_0)$ 
6:     for  $p_i$  in  $\mathcal{P}_o$  do
7:        $\vec{v}_c \leftarrow \vec{L}_o$   $\triangleright$  Get direction vector of line  $\vec{L}_o$ .
8:        $\vec{v}_i = p_c - p_i$   $\triangleright$  Get direction vector of line  $\vec{p}_c p_i$ .
9:       if  $\langle \vec{v}_i, \vec{v}_c \rangle \geq 0$  then  $\triangleright \langle \cdot, \cdot \rangle$  denotes scalar product.
10:         $\mathcal{P}_+ \leftarrow \mathcal{P}_+ \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_+$ .
11:      else
12:         $\mathcal{P}_- \leftarrow \mathcal{P}_- \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_-$ .
13:      end if
14:    end for
15:    if  $p_c == p_0$  &  $\otimes == \mathcal{A}$  &  $\mathcal{P}_- \neq \emptyset$  then
16:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_-} \|p_l, p_0\|$   $\triangleright \|\cdot, \cdot\|$  is Euclidean-distance.
17:    else if  $p_c == p_0$  &  $\otimes == \mathcal{B}$  &  $\mathcal{P}_+ \neq \emptyset$  then
18:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_+} \|p_l, p_0\|$ 
19:    end if  $\triangleright$  Ensure ordering direction at the first point.
20:    if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_+$  &  $\mathcal{P}_- \neq \emptyset$  then
21:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_-} \|p_o, p_0\|$ 
22:    else if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_-$  &  $\mathcal{P}_+ \neq \emptyset$  then
23:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_+} \|p_o, p_0\|$ 
24:    end if
25:     $d_c = d_o + d_\epsilon$   $\triangleright$  Increase collecting distance (no satisfied  $p_o$ ).
26:    if  $p_o \neq \emptyset$  &  $p_o \notin \mathcal{Q}_O$  then
27:       $\mathcal{Q}_O \leftarrow \mathcal{Q}_O \cup p_o$   $\triangleright$  Insert  $p_o$  to  $\mathcal{Q}_O$ .
28:       $p_l = p_c$   $\triangleright$  Update  $p_l$ .
29:       $p_c = p_o$   $\triangleright$  Update  $p_c$ .
30:       $d_c = d_0$   $\triangleright$  Reset collecting distance.
31:    end if
32:  end while
33:  return  $\mathcal{Q}_O$ 
34: end function

```

Sorting Unarranged 2D Points

Moving Least Square Method



Algorithm 3 Ordering points with a given direction $\otimes$ at point p_0 , given initial collecting distance d_0 , increment collecting distance d_ϵ , maximum break distance d_B .

Require: $\otimes, p_0, d_0, d_\epsilon, d_B, \mathcal{P}, d_M, \rho_0$
Ensure: $\mathcal{Q}_O$

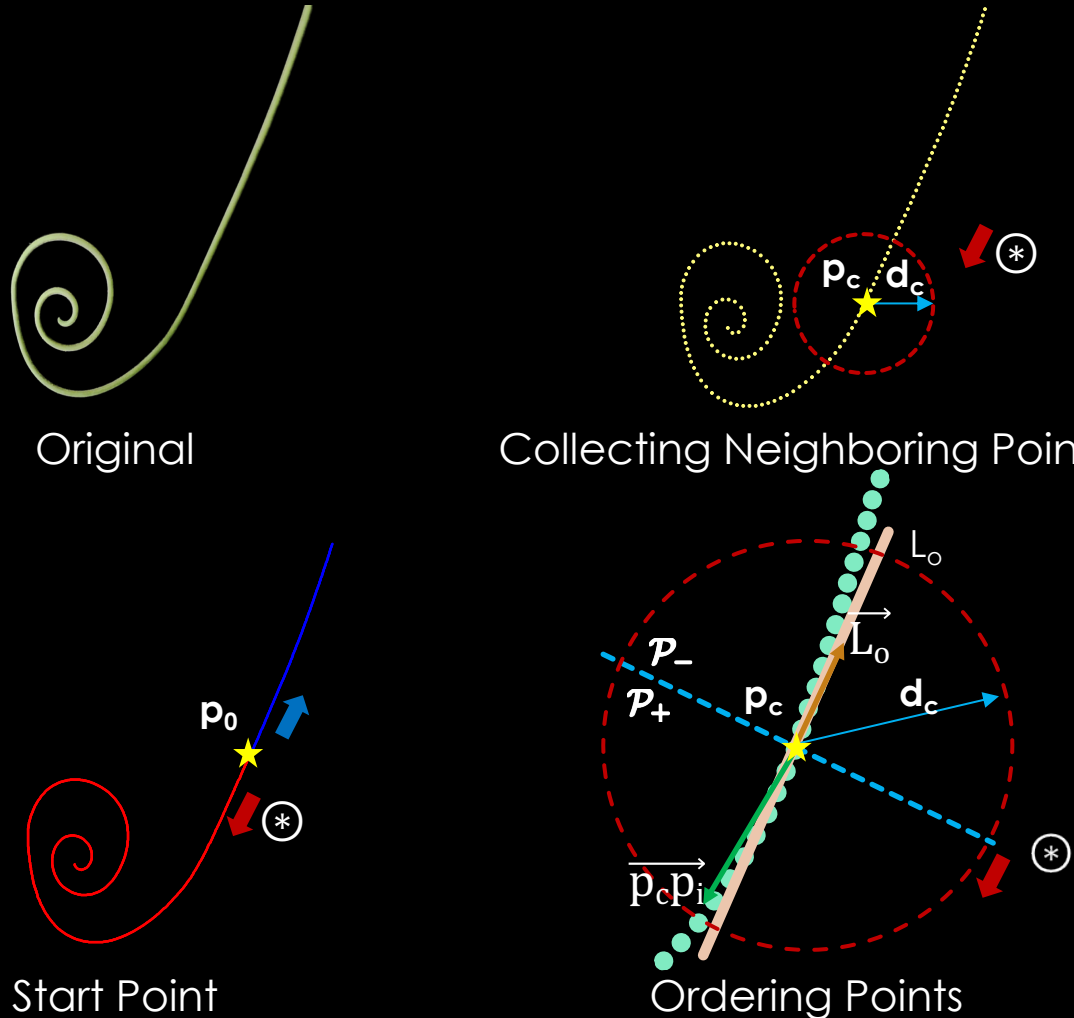
```

1: function OrderingPoints( $\otimes, p_0, \mathcal{P}$ )
2:    $p_c = p_0, d_c = d_0, \mathcal{Q}_O = \emptyset, p_l = \emptyset$   $\triangleright$  Initialization.
3:   while  $d_c \leq d_B$  do
4:      $\mathcal{P}_+, \mathcal{P}_- = \emptyset, p_o = \emptyset$   $\triangleright$  Reset as empty.
5:      $\mathcal{P}_o, d_o, \mathcal{L}_o \leftarrow \text{CollectNeighbouringPoints}(p_c, d_c, \mathcal{P}, d_M, \rho_0)$ 
6:     for  $p_i$  in  $\mathcal{P}_o$  do
7:        $\vec{v}_c \leftarrow \vec{L}_o$   $\triangleright$  Get direction vector of line  $\vec{L}_o$ .
8:        $\vec{v}_i = p_c - p_i$   $\triangleright$  Get direction vector of line  $\vec{p}_c p_i$ .
9:       if  $\langle \vec{v}_i, \vec{v}_c \rangle \geq 0$  then  $\triangleright \langle \cdot, \cdot \rangle$  denotes scalar product.
10:         $\mathcal{P}_+ \leftarrow \mathcal{P}_+ \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_+$ .
11:      else
12:         $\mathcal{P}_- \leftarrow \mathcal{P}_- \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_-$ .
13:      end if
14:    end for
15:    if  $p_c == p_0$  &  $\otimes == \mathcal{A}$  &  $\mathcal{P}_- \neq \emptyset$  then
16:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_-} \|p_l, p_0\|$   $\triangleright \|\cdot, \cdot\|$  is Euclidean-distance.
17:    else if  $p_c == p_0$  &  $\otimes == \mathcal{B}$  &  $\mathcal{P}_+ \neq \emptyset$  then
18:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_+} \|p_l, p_0\|$ 
19:    end if  $\triangleright$  Ensure ordering direction at the first point.
20:    if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_+$  &  $\mathcal{P}_- \neq \emptyset$  then
21:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_-} \|p_o, p_0\|$ 
22:    else if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_-$  &  $\mathcal{P}_+ \neq \emptyset$  then
23:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_+} \|p_o, p_0\|$ 
24:    end if
25:     $d_c = d_o + d_\epsilon$   $\triangleright$  Increase collecting distance (no satisfied  $p_o$ ).
26:    if  $p_o \neq \emptyset$  &  $p_o \notin \mathcal{Q}_O$  then
27:       $\mathcal{Q}_O \leftarrow \mathcal{Q}_O \cup p_o$   $\triangleright$  Insert  $p_o$  to  $\mathcal{Q}_O$ .
28:       $p_l = p_c$   $\triangleright$  Update  $p_l$ .
29:       $p_c = p_o$   $\triangleright$  Update  $p_c$ .
30:       $d_c = d_0$   $\triangleright$  Reset collecting distance.
31:    end if
32:  end while
33:  return  $\mathcal{Q}_O$ 
34: end function

```

Sorting Unarranged 2D Points

Moving Least Square Method



Algorithm 3 Ordering points with a given direction $\otimes$ at point p_0 , given initial collecting distance d_0 , increment collecting distance d_ϵ , maximum break distance d_B .

Require: $\otimes, p_0, d_0, d_\epsilon, d_B, \mathcal{P}, d_M, \rho_0$

Ensure: $\mathcal{Q}_O$

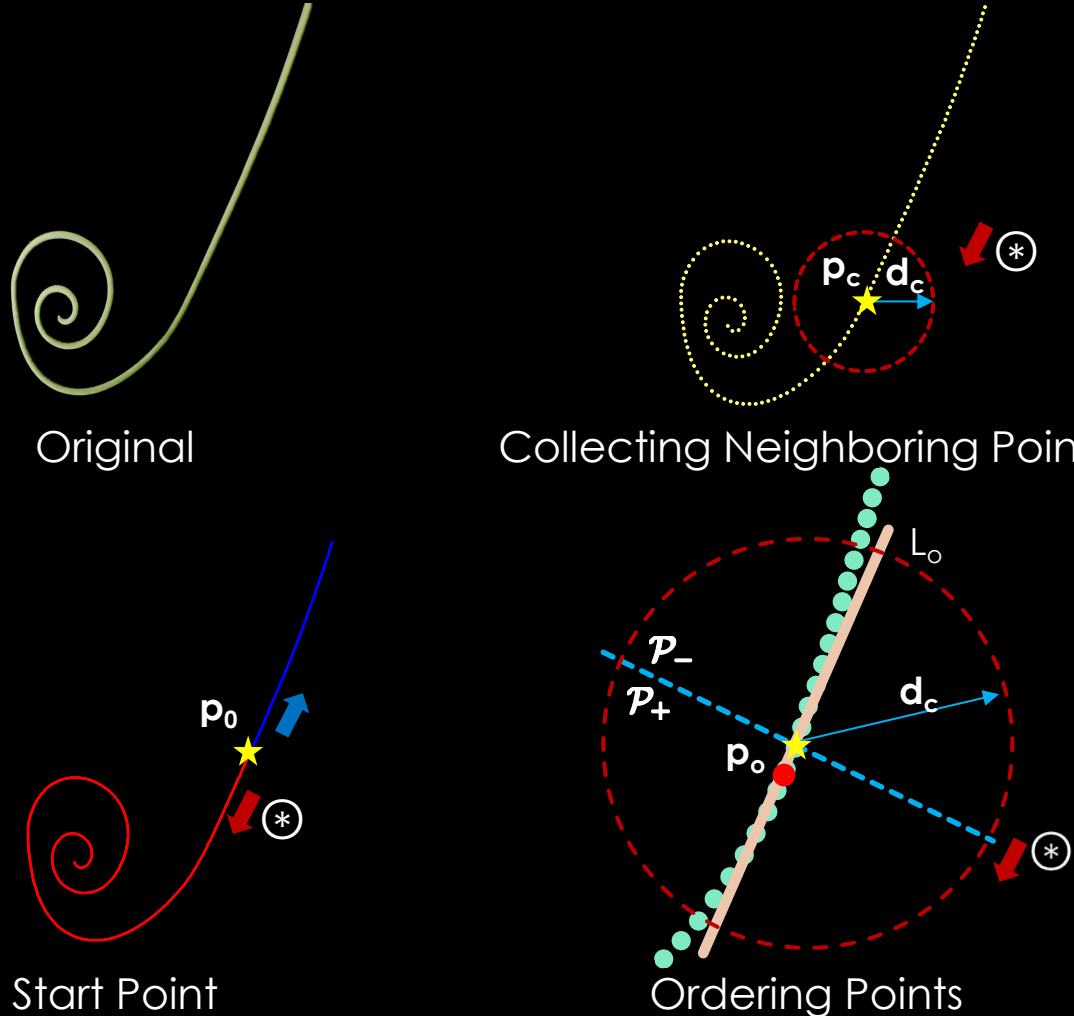
```

1: function OrderingPoints( $\otimes, p_0, \mathcal{P}$ )
2:    $p_c = p_0, d_c = d_0, \mathcal{Q}_O = \emptyset, p_l = \emptyset$   $\triangleright$  Initialization.
3:   while  $d_c \leq d_B$  do
4:      $\mathcal{P}_+, \mathcal{P}_- = \emptyset, p_o = \emptyset$   $\triangleright$  Reset as empty.
5:      $\mathcal{P}_o, d_o, \overline{\mathcal{L}}_o \leftarrow \text{CollectNeighbouringPoints}(p_c, d_c, \mathcal{P}, d_M, \rho_0)$ 
6:     for  $p_i$  in  $\mathcal{P}_o$  do
7:        $\overline{v}_c \leftarrow \overline{\mathcal{L}}_o$   $\triangleright$  Get direction vector of line  $\overline{\mathcal{L}}_o$ .
8:        $\overline{v}_i = p_c - p_i$   $\triangleright$  Get direction vector of line  $\overline{p_c p_i}$ .
9:       if  $\langle \overline{v}_i, \overline{v}_c \rangle \geq 0$  then  $\triangleright \langle \cdot, \cdot \rangle$  denotes scalar product.
10:         $\mathcal{P}_+ \leftarrow \mathcal{P}_+ \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_+$ .
11:      else
12:         $\mathcal{P}_- \leftarrow \mathcal{P}_- \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_-$ .
13:      end if
14:    end for
15:    if  $p_c == p_0$  &  $\otimes == \mathcal{A}$  &  $\mathcal{P}_- \neq \emptyset$  then
16:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_-} \|p_l, p_0\|$   $\triangleright \|\cdot, \cdot\|$  is Euclidean-distance.
17:    else if  $p_c == p_0$  &  $\otimes == \mathcal{B}$  &  $\mathcal{P}_+ \neq \emptyset$  then
18:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_+} \|p_l, p_0\|$ 
19:    end if  $\triangleright$  Ensure ordering direction at the first point.
20:    if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_+$  &  $\mathcal{P}_- \neq \emptyset$  then
21:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_-} \|p_o, p_0\|$ 
22:    else if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_-$  &  $\mathcal{P}_+ \neq \emptyset$  then
23:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_+} \|p_o, p_0\|$ 
24:    end if
25:     $d_c = d_o + d_\epsilon$   $\triangleright$  Increase collecting distance (no satisfied  $p_o$ ).
26:    if  $p_o \neq \emptyset$  &  $p_o \notin \mathcal{Q}_O$  then
27:       $\mathcal{Q}_O \leftarrow \mathcal{Q}_O \cup p_o$   $\triangleright$  Insert  $p_o$  to  $\mathcal{Q}_O$ .
28:       $p_l = p_c$   $\triangleright$  Update  $p_l$ .
29:       $p_c = p_o$   $\triangleright$  Update  $p_c$ .
30:       $d_c = d_0$   $\triangleright$  Reset collecting distance.
31:    end if
32:  end while
33:  return  $\mathcal{Q}_O$ 
34: end function

```


Sorting Unarranged 2D Points

Moving Least Square Method



Algorithm 3 Ordering points with a given direction $\otimes$ at point p_0 , given initial collecting distance d_0 , increment collecting distance d_ϵ , maximum break distance d_B .

Require: $\otimes, p_0, d_0, d_\epsilon, d_B, \mathcal{P}, d_M, \rho_0$

Ensure: $\mathcal{Q}_O$

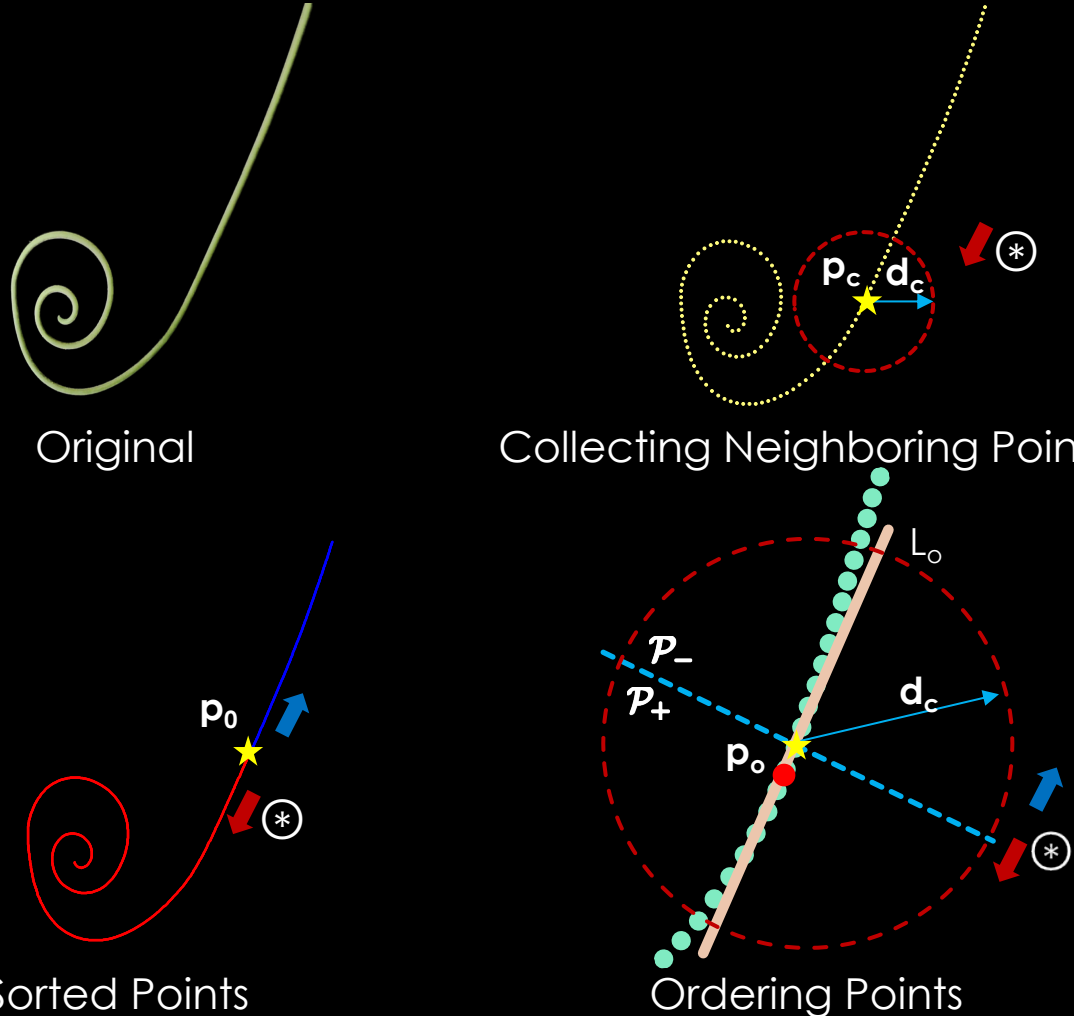
```

1: function OrderingPoints( $\otimes, p_0, \mathcal{P}$ )
2:    $p_c = p_0, d_c = d_0, \mathcal{Q}_O = \emptyset, p_l = \emptyset$   $\triangleright$  Initialization.
3:   while  $d_c \leq d_B$  do
4:      $\mathcal{P}_+, \mathcal{P}_- = \emptyset, p_o = \emptyset$   $\triangleright$  Reset as empty.
5:      $\mathcal{P}_o, d_o, \overline{\mathcal{L}}_o \leftarrow \text{CollectNeighbouringPoints}(p_c, d_c, \mathcal{P},$ 
       $d_M, \rho_0)$ 
6:     for  $p_i$  in  $\mathcal{P}_o$  do
7:        $\overline{v}_c \leftarrow \overline{\mathcal{L}}_o$   $\triangleright$  Get direction vector of line  $\overline{\mathcal{L}}_o$ .
8:        $\overline{v}_i = p_c - p_i$   $\triangleright$  Get direction vector of line  $\overline{p_c p_i}$ .
9:       if  $\langle \overline{v}_i, \overline{v}_c \rangle \geq 0$  then  $\triangleright \langle \cdot, \cdot \rangle$  denotes scalar product.
10:         $\mathcal{P}_+ \leftarrow \mathcal{P}_+ \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_+$ .
11:      else
12:         $\mathcal{P}_- \leftarrow \mathcal{P}_- \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_-$ .
13:      end if
14:    end for
15:    if  $p_c == p_0$  &  $\otimes == \mathcal{A}$  &  $\mathcal{P}_- \neq \emptyset$  then
16:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_-} \|p_l, p_0\|$   $\triangleright \|\cdot, \cdot\|$  is Euclidean-distance.
17:    else if  $p_c == p_0$  &  $\otimes == \mathcal{B}$  &  $\mathcal{P}_+ \neq \emptyset$  then
18:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_+} \|p_l, p_0\|$ 
19:    end if  $\triangleright$  Ensure ordering direction at the first point.
20:    if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_+$  &  $\mathcal{P}_- \neq \emptyset$  then
21:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_-} \|p_o, p_0\|$ 
22:    else if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_-$  &  $\mathcal{P}_+ \neq \emptyset$  then
23:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_+} \|p_o, p_0\|$ 
24:    end if
25:     $d_c = d_o + d_\epsilon$   $\triangleright$  Increase collecting distance (no satisfied  $p_o$ ).
26:    if  $p_o \neq \emptyset$  &  $p_o \notin \mathcal{Q}_O$  then
27:       $\mathcal{Q}_O \leftarrow \mathcal{Q}_O \cup p_o$   $\triangleright$  Insert  $p_o$  to  $\mathcal{Q}_O$ .
28:       $p_l = p_c$   $\triangleright$  Update  $p_l$ .
29:       $p_c = p_o$   $\triangleright$  Update  $p_c$ .
30:       $d_c = d_0$   $\triangleright$  Reset collecting distance.
31:    end if
32:  end while
33:  return  $\mathcal{Q}_O$ 
34: end function

```

Sorting Unarranged 2D Points

Moving Least Square Method



Algorithm 3 Ordering points with a given direction $\otimes$ at point p_0 , given initial collecting distance d_0 , increment collecting distance d_ϵ , maximum break distance d_B .

Require: $\otimes, p_0, d_0, d_\epsilon, d_B, \mathcal{P}, d_M, \rho_0$
Ensure: $\mathcal{Q}_O$

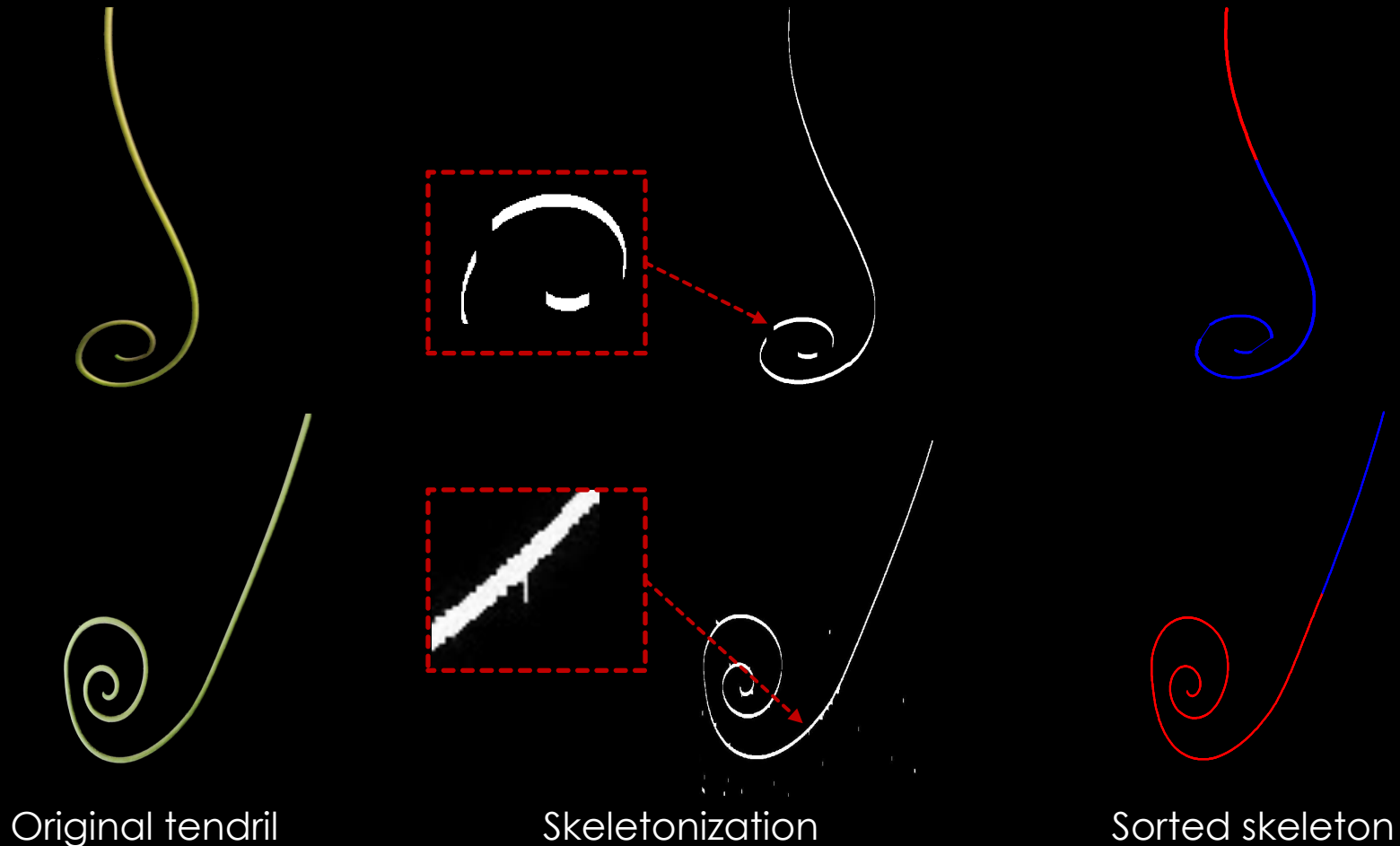
```

1: function OrderingPoints( $\otimes, p_0, \mathcal{P}$ )
2:    $p_c = p_0, d_c = d_0, \mathcal{Q}_O = \emptyset, p_l = \emptyset$   $\triangleright$  Initialization.
3:   while  $d_c \leq d_B$  do
4:      $\mathcal{P}_+, \mathcal{P}_- = \emptyset, p_o = \emptyset$   $\triangleright$  Reset as empty.
5:      $\mathcal{P}_o, d_o, \mathcal{L}_o \leftarrow \text{CollectNeighbouringPoints}(p_c, d_c, \mathcal{P}, d_M, \rho_0)$ 
6:     for  $p_i$  in  $\mathcal{P}_o$  do
7:        $\vec{v}_c \leftarrow \mathcal{L}_o$   $\triangleright$  Get direction vector of line  $\mathcal{L}_o$ .
8:        $\vec{v}_i = p_c - p_i$   $\triangleright$  Get direction vector of line  $\overline{p_c p_i}$ .
9:       if  $\langle \vec{v}_i, \vec{v}_c \rangle \geq 0$  then  $\triangleright \langle \cdot, \cdot \rangle$  denotes scalar product.
10:         $\mathcal{P}_+ \leftarrow \mathcal{P}_+ \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_+$ .
11:      else
12:         $\mathcal{P}_- \leftarrow \mathcal{P}_- \cup p_i$   $\triangleright$  Insert  $p_i$  to  $\mathcal{P}_-$ .
13:      end if
14:    end for
15:    if  $p_c == p_0$  &  $\otimes == \mathcal{A}$  &  $\mathcal{P}_- \neq \emptyset$  then
16:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_-} \|p_l, p_0\|$   $\triangleright \|\cdot, \cdot\|$  is Euclidean-distance.
17:    else if  $p_c == p_0$  &  $\otimes == \mathcal{B}$  &  $\mathcal{P}_+ \neq \emptyset$  then
18:       $p_l \leftarrow \text{argmin}_{p_l \in \mathcal{P}_+} \|p_l, p_0\|$ 
19:    end if  $\triangleright$  Ensure ordering direction at the first point.
20:    if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_+$  &  $\mathcal{P}_- \neq \emptyset$  then
21:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_-} \|p_o, p_0\|$ 
22:    else if  $p_l \neq \emptyset$  &  $p_l \in \mathcal{P}_-$  &  $\mathcal{P}_+ \neq \emptyset$  then
23:       $p_o \leftarrow \text{argmin}_{p_o \in \mathcal{P}_+} \|p_o, p_0\|$ 
24:    end if
25:     $d_c = d_o + d_\epsilon$   $\triangleright$  Increase collecting distance (no satisfied  $p_o$ ).
26:    if  $p_o \neq \emptyset$  &  $p_o \notin \mathcal{Q}_O$  then
27:       $\mathcal{Q}_O \leftarrow \mathcal{Q}_O \cup p_o$   $\triangleright$  Insert  $p_o$  to  $\mathcal{Q}_O$ .
28:       $p_l = p_c$   $\triangleright$  Update  $p_l$ .
29:       $p_c = p_o$   $\triangleright$  Update  $p_c$ .
30:       $d_c = d_0$   $\triangleright$  Reset collecting distance.
31:    end if
32:  end while
33:  return  $\mathcal{Q}_O$ 
34: end function

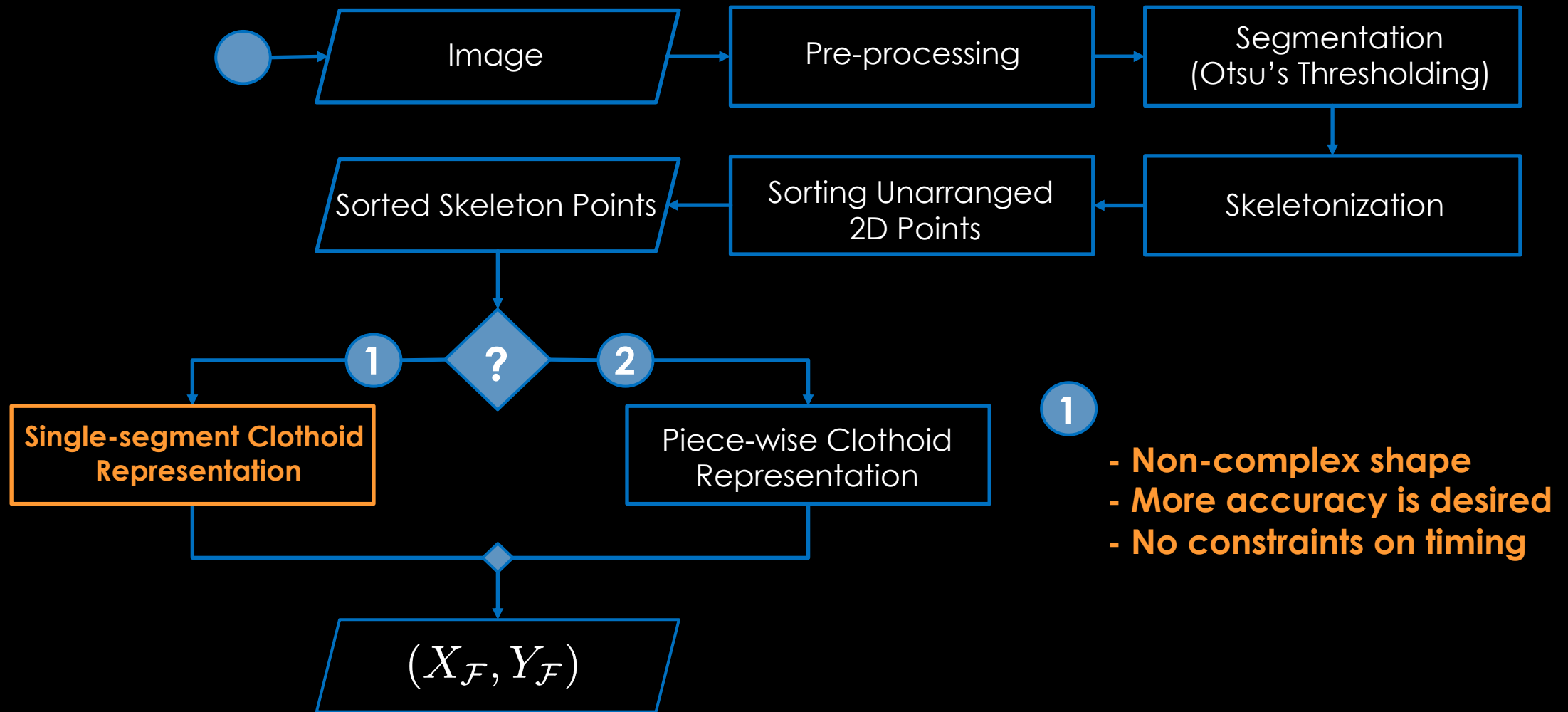
```

Results

Applicable even in the presence of discontinuities, excessive noise or small branches



Flowchart



Single-Segment Clothoid Representation

An optimization problem to identify one segment clothoid

$$\begin{aligned} \min_{\Theta} \quad & \frac{1}{2N} [(X_I - X_F)^\top (X_I - X_F) + (Y_I - Y_F)^\top (Y_I - Y_F)] \\ \text{s.t.} \quad & \Theta = \{w_s, h_s, \beta, k_1, l_a, l_b\} \\ & \Theta_{\min} \leq \Theta \leq \Theta_{\max} \end{aligned}$$

width scale	w_s	sorted skeleton	(X_I, Y_I)
height scale	h_s	fitted clothoid	(X_F, Y_F)
rotation angle	β		
curvature	k_1		
start length	l_a		
end length	l_b		

Single-Segment Clothoid Representation

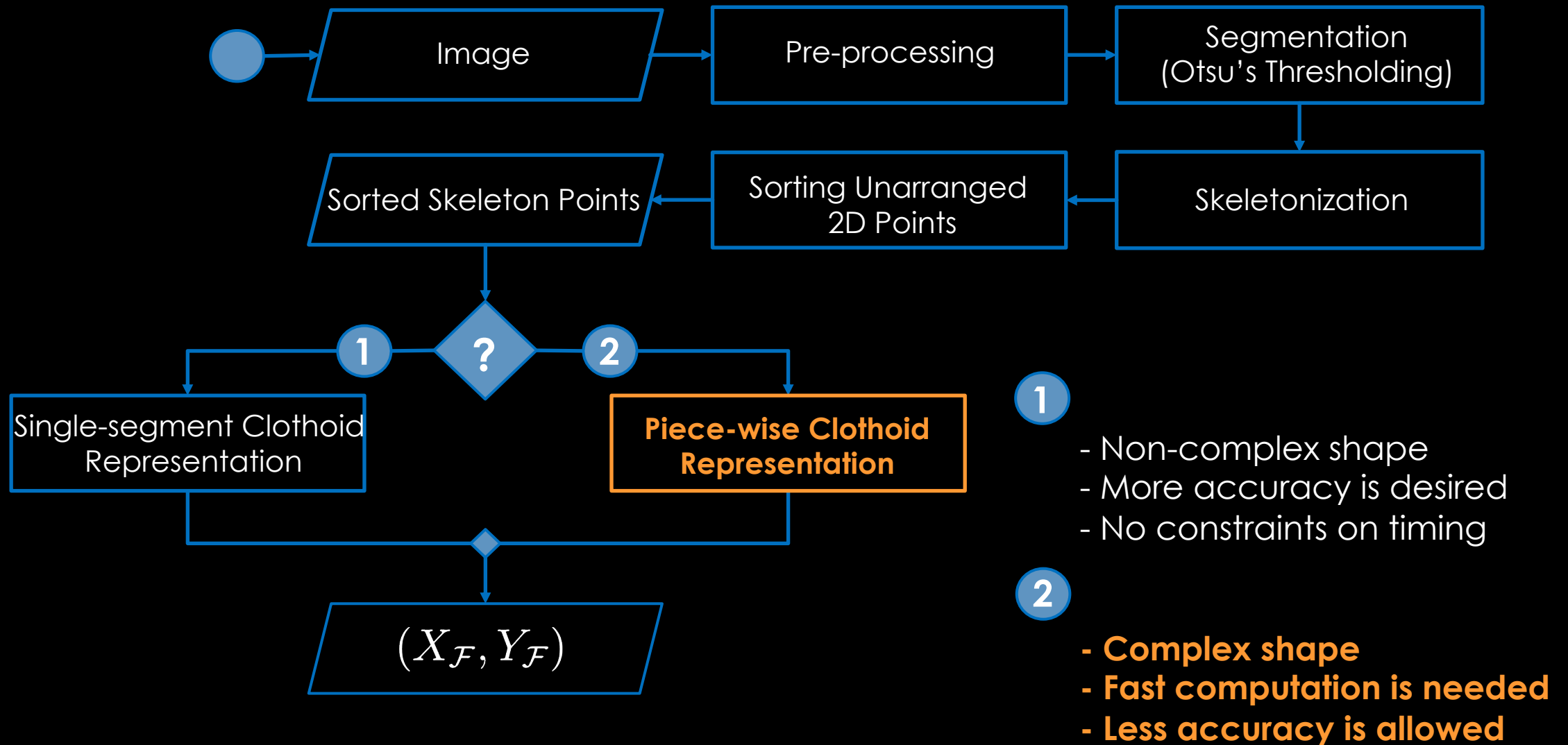
An optimization problem to identify one segment clothoid

$$\begin{aligned} \min_{\Theta} \quad & \frac{1}{2N} [(X_I - X_F)^\top (X_I - X_F) + (Y_I - Y_F)^\top (Y_I - Y_F)] \\ \text{s.t.} \quad & \Theta = \{w_s, h_s, \beta, k_1, l_a, l_b\} \\ & \Theta_{\min} \leq \Theta \leq \Theta_{\max} \end{aligned}$$

width scale	w_s	sorted skeleton	(X_I, Y_I)
height scale	h_s	fitted clothoid	(X_F, Y_F)
rotation angle	β		
curvature	k_1		
start length	l_a		
end length	l_b		

Global Optimal
Gradient Descent

Flowchart



Piece-wise Clothoid Representation

G¹ Hermite interpolation to guarantee a minimization of error and smooth transition of adjacent segments

G¹ Hermite interpolation Given two points (x_0, y_0) and (x_1, y_1) and two angles α_0 and α_1 , find a curve segment of **Eq.2** that satisfies

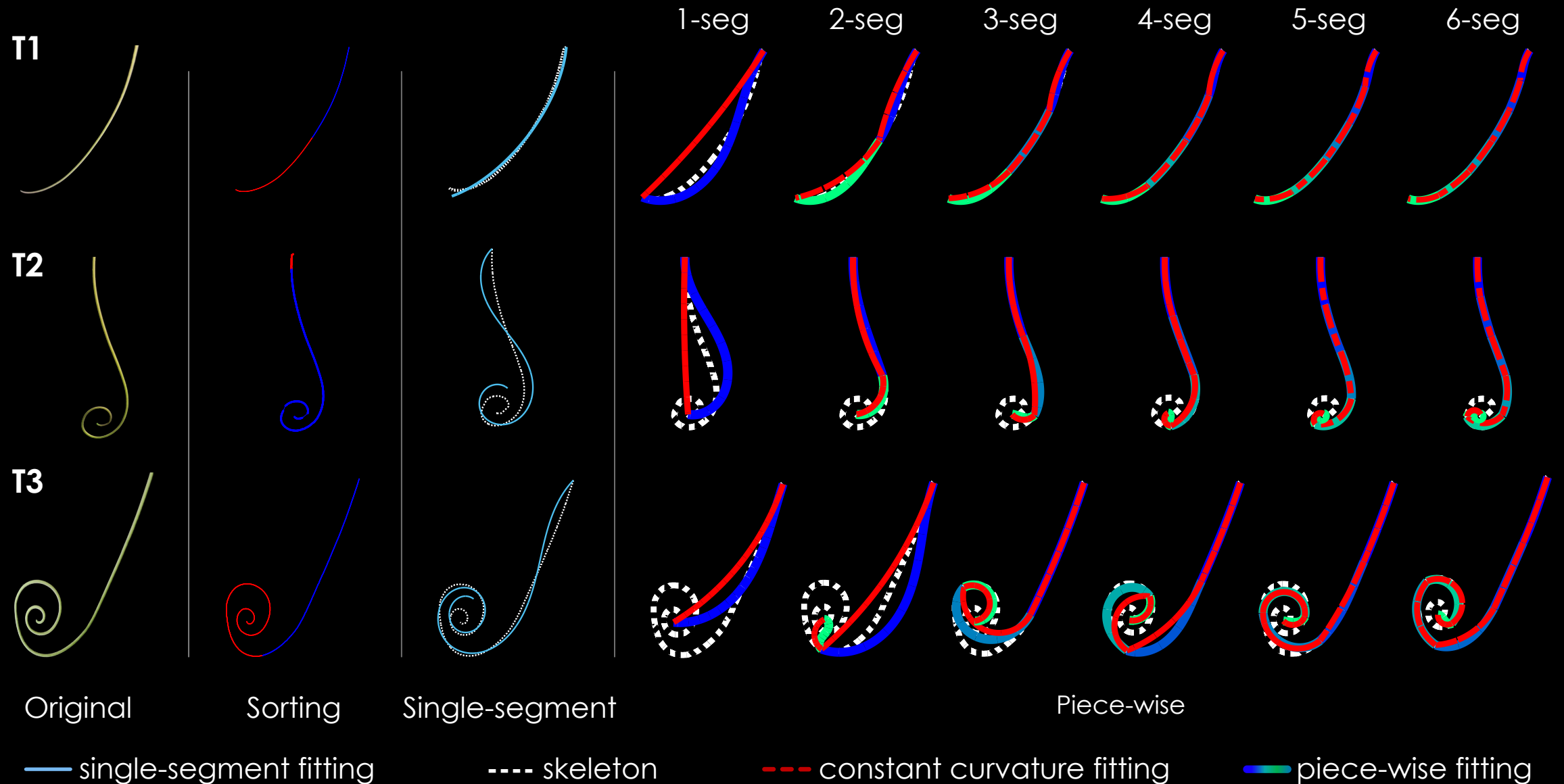
$$\begin{aligned}x(0) &= x_0, \quad y(0) = y_0, \quad (x'(0), y'(0)) = (\cos\alpha_0, \sin\alpha_0) \\x(L) &= x_1, \quad y(L) = y_1, \quad (x'(L), y'(L)) = (\cos\alpha_1, \sin\alpha_1)\end{aligned}$$

where $L > 0$ is the length of the curve segment.

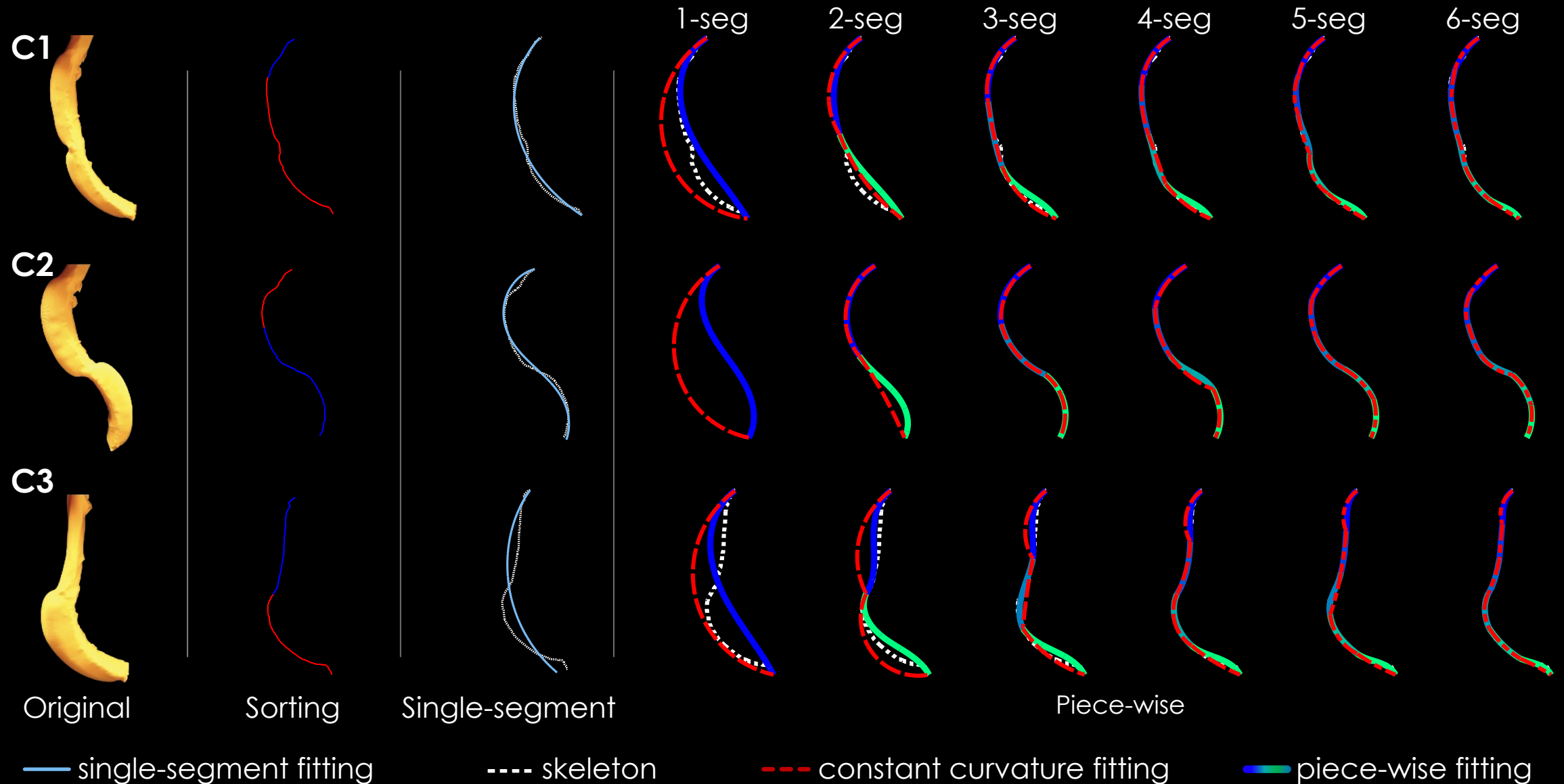
Fresnel Integrals finds (Eq.1) solution

$$\begin{aligned}x(s) &= x_0 + \int_0^s \cos\left(\frac{1}{2}k_1\xi^2 + k_2\xi + \alpha_0\right) d\xi = x_0 + s\mathbb{X}_0(k_1s^2, k_2s, \alpha_0) \\y(s) &= y_0 + \int_0^s \sin\left(\frac{1}{2}k_1\xi^2 + k_2\xi + \alpha_0\right) d\xi = y_0 + s\mathbb{Y}_0(k_1s^2, k_2s, \alpha_0)\end{aligned}\tag{Eq.2}$$

Results – Natural Tendrils



Results – Continuum Structures



Results

1. Single-segment with high accuracy ($R^2 \geq 0.968$) but time-consuming
2. Piece-wise clothoid with relative low accuracy but much faster ($t \sim 1s$), suitable for real time

R^2 , Time	T1	Time T1 (s)	T2	Time T2 (s)	T3	Time T3 (s)	C1	Time C1 (s)	C2	Time C2 (s)	C3	Time C3 (s)
Method 1	0.998	131.7	0.968	91.82	0.989	144.2	0.998	114.44	0.997	132.0	0.980	132.4
Method 2	0.986	0.128	0.697	0.120	-0.15	0.136	0.978	0.106	0.958	0.106	0.929	0.102

Table. Comparison of fitting error and execution time for two methods (only considering one segment case)

T1,T2,T3: natural tendrils

C1,C2,C3: continuum artifacts

Conclusions

1. The **MLS sorting algorithm** is effective and robust for sorting unordered thin point cloud
2. Single-segment clothoid with curve optimization is better for **simple shape and no constraint on timing**
3. Piece-wise clothoid with G^1 Hermite-fitting is better for **complicated shape and fast computation**
4. Clothoid is better than constant curvature in the presence of the **transition between variable curvatures**, has a great potential for long soft robots with curling behavior

Next steps

1. Automatic piece-wise clothoid representation for tendril-like curling shapes
2. Extend the analysis from 2D to 3D case and relate the shape parameters to control parameters for continuum soft tendril-like robots



Thank you

jie.fan@iit.it
Jie.fan@santannapisa.it

<https://bsr.iit.it/>



@BSRiit

BSR Bioinspired Soft Robotics lab